



Software Engineering Foundations

ENTERPRISE MOBILE APPLICATION DEVELOPMENT BOOTCAMP

PHASE 01

Software Engineering Foundations for Mobile

Phase 01 - Three Modules at a Glance

Module	Schedule	Core Skills
Module 1 Software Engineering Principles	Day 1 & 2	SDLC · Agile/Scrum · Platform strategy · SOLID (all 5) · Clean code · Technical debt · User stories
Module 2 Source Control and Team Development	Day 3 & 4	Git internals · Core operations · GitFlow / TBD / GitHub Flow · Conventional Commits · PRs · Code review · Conflicts · Advanced Git
Module 3 Problem Solving and Engineering Thinking	Day 5	Decomposition · Technical communication · Estimation · 7-step debugging · Root cause analysis · Architecture whiteboarding



Modern Software Engineering Principles

Module 1

The Software Development Lifecycle

THE PROBLEM

Why can't we just start coding?

Every project that skips planning pays for it in rework, missed requirements, and production defects.

The Seven SDLC Phases

Phase	What Happens	What Gets Skipped = What Goes Wrong
Planning	Scope, resources, risks, feasibility	Scope creep, missed deadlines, wrong team size
Requirements	What the software must do (functional) and how (NFRs)	Building the wrong thing. Rework.
Design	Architecture, API contracts, data models, UI flows	Incompatible components. Integration surprises.
Development	Code, review, integrate	Without prior phases: chaos
Testing	Correctness, performance, security, usability	Production defects. Loss of user trust.
Deployment	Release through a controlled process, CI/CD	Unreproducible deployments. Rollback failures.
Maintenance	Monitor, fix, iterate - 60–80% of lifecycle cost	Technical debt accumulation. Slow teams.



SDLC Models - Which Fits Your Project?

- Waterfall: sequential, each phase completes before the next - rigid, but right for fixed regulatory specs
- Iterative: phases repeat in cycles - earlier feedback, some flexibility
- Agile / Scrum: two-week sprints, working software every cycle - dominant model for enterprise mobile
- Spiral: risk-driven iterations - used in safety-critical systems (medical, aerospace)
 - Is a meta-model, combining features of Waterfall, Agile, and Iterative
- This program uses Agile/Scrum throughout - the model you will use in every Phase lab



Which SDLC Phase Gets Skipped Most?

Skill Check ✓

Your Turn

Which phase do you think teams skip most often in real projects? What happens as a result?



Agile and the Scrum Framework

THE AGILE MANIFESTO VALUES (2001)

Individuals and interactions over processes and tools.

Working software over comprehensive documentation.

Customer collaboration over contract negotiation.

Responding to change over following a plan.

The items on the right have value - we just value the items on the left more.

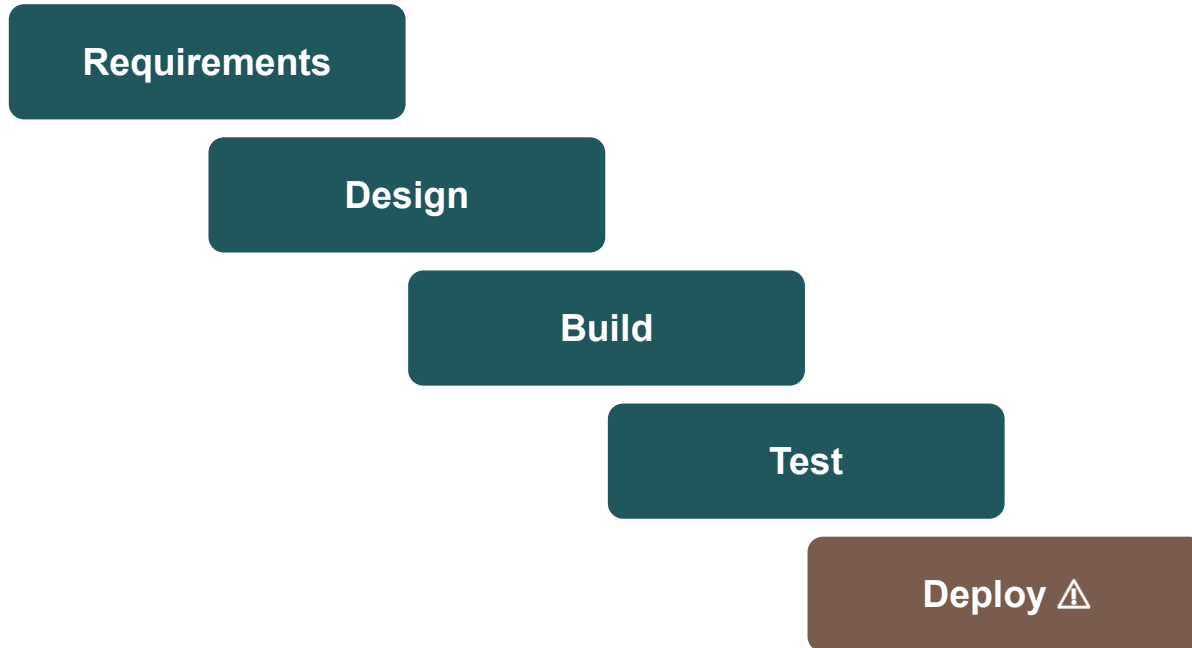
The 12 Agile Principles - Operational Meaning

#	Principle	What It Means in Mobile Engineering
1	Early and continuous delivery of working software	Builds to TestFlight/Firebase every Sprint - not demos, not prototypes
9	Technical excellence and good design enhance agility	SOLID, clean code, tests - not opposed to speed, they enable it
10	Simplicity - maximizing the work NOT done	Build what's needed now. Over-engineering is as dangerous as under-engineering
12	Reflect and adjust at regular intervals	The retrospective operationalizes this - it's not optional



Waterfall Cascades. Scrum Loops.

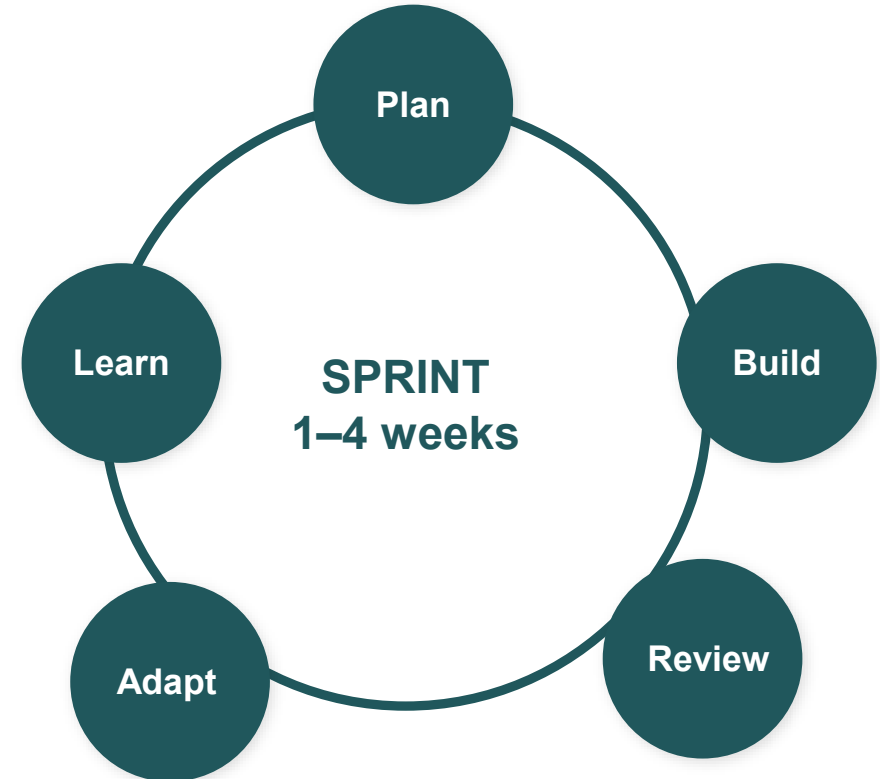
WATERFALL - value arrives at the end



⚠ Feedback arrives here - months later

Change is expensive. Risk is discovered late. Stakeholders see the product once.

SCRUM - value arrives every Sprint



✓ Working product + feedback every cycle

Change is cheap. Risk surfaces early. Stakeholders see real results continuously.



Scrum Roles - What They Are and Are Not

Role	Core Responsibility	What They Are NOT
Product Owner	Owns and orders the backlog. Single point of contact for business requirements.	Not a requirements gatherer. Not someone who can be unavailable for days.
Scrum Master	Serves the team. Removes impediments. Protects the process. Coaches.	Not a project manager. Not a task-assigner. Not a secretary.
Developers	Create the increment. Collectively accountable for quality and delivery.	Not just coders - everyone who contributes to the increment.



Three Accountabilities. One Team. Zero Hierarchy.



Product Owner

Owns **VALUE**

- Orders the Product Backlog
- Makes prioritization decisions
- Carries the product vision
- One person - not a committee



Scrum Master

Owns **EFFECTIVENESS**

- Coaches the team on Scrum
- Removes impediments
- Shields team from interference
- Leads by serving - not directing



Developers

Own the **HOW**

- Everyone building the Increment
- Self-managing - no task assignment
- Collectively own quality (DoD)
- Cross-functional by design



The Product Owner: Value Decision-Maker



The PO is accountable for...

- Maximizing the value of the product
- Owning and ordering the Product Backlog
- Deciding what to build next - by value and risk
- Communicating the product vision and goal
- Being available to the team for fast answers



The PO is not...

- A requirements writer who hands off a spec
- A committee or a rotating seat
- A task assigner directing the Developers
- A proxy who must escalate every decision
- Optional - an absent PO starves the team



The Scrum Master: Coaching, Not Policing



Serves the team

Removes impediments blocking flow; creates the conditions for focus



Serves the Product Owner

Helps with backlog techniques and stakeholder collaboration



Serves the organization

Coaches leadership on how to work with - not over - Scrum teams

Never: assigns work · manages people · reports team status upward

When the SM starts doing these, self-management dies - and Scrum with it.



“Who does status reporting, then?”

Nobody - because the artifacts and events ARE the status. The Sprint Review shows working product. The backlog shows what's done and what's next. Transparency by default, not by report.



Developers: Everyone Who Builds the Increment

“Developer” is not a job title in Scrum - it's anyone whose work creates the Increment: engineers, testers, designers, analysts, writers, data scientists.



Cross-functional accountability

The team collectively owns delivery - no “not my job” inside the Sprint



They create the Sprint plan

The PO says what and why. The Developers decide how - and estimate it



Quality lives with the team

The Definition of Done is enforced by the people doing the work



Self-management, in reality

Not the absence of management - management that lives inside the team rather than above it



Three Artifacts, Three Commitments



WHAT + WHY

Product Backlog

The single, ordered list of everything that might improve the product. Never complete - it evolves as the team learns.

Commitment: Product Goal



THIS SPRINT

Sprint Backlog

The Sprint Goal, the selected items, and the team's plan. Owned by the Developers and updated daily.

Commitment: Sprint Goal



WHAT'S REAL

Increment

All completed work - usable, meeting the quality bar. If it doesn't meet the DoD, it isn't done.

Commitment: Definition of Done



The Product Backlog: What Belongs, What Doesn't

Belongs in the Product Backlog

- ✓ Features and user stories
- ✓ Defects and technical debt
- ✓ Compliance and regulatory items
- ✓ Research spikes and experiments
- ✓ Improvements surfaced by Retros

Doesn't belong

- ✗ Task-level breakdowns (that's the Sprint Backlog)
- ✗ Items no one intends to ever do
- ✗ Duplicate wish lists from every stakeholder
- ✗ Hidden side-channel work



Order for value and risk - not for politics.

High value + high risk items rise to the top: learn the hard things early, while change is still cheap.



Sprint Backlog



Sprint Backlog

A forecast, not a contract

“We believe we can do this” - updated daily as the team learns

Owned entirely by the team

No outside party adds, removes, or verifies items

Transparency, not control

Visible to everyone; managed by no one above the team



Increment - Delivered Working Product



Definition of Done

“Done” is the most dangerous word in delivery. Coded but not tested? Tested but not deployed? Deployed but not accepted?

The DoD is a quality contract:

- One shared standard for “complete”
- Applied to every item, every Sprint
- If it doesn't meet the DoD - it is not done. Period.



Writing User Stories and Acceptance Criteria

The INVEST Criteria - What Makes a Good Story

Letter	Characteristic	What it Means
I	Independent	The story can be developed without depending on other stories
N	Negotiable	The story is a placeholder for conversation, not a detailed contract
V	Valuable	The story delivers value to a user, customer, or stakeholder
E	Estimable	The team understands it well enough to estimate the effort
S	Small	The story is small enough to fit comfortably within a sprint
T	Testable	The team can verify whether the story is done



From BRD Statement to User Story

BRD STATEMENT

"The system shall allow users to filter search results by date range, category, and price."



USER STORY

As a **customer**, I want to **filter my search results by date range, category, and price**, so that **I can select the right product for my needs**.

The anatomy - and why each part matters

WHO

As a [type of user]

Keeps a real human in the frame - not "the system"

WHAT

I want to [action]

The capability, stated in user language

WHY

So that [benefit]

The value test - if you can't fill this in, ask why it's in the backlog



Writing Acceptance Criteria

- Acceptance criteria define the conditions under which a user story is considered complete
- They should be included in a user story that is ready for development
 - Definition of Ready
- The Gherkin syntax (Given/When/Then) structures them as testable scenarios:
 - Given: The preconditions or context (the world's state before the action)
 - When: The action the user takes or the event that occurs
 - Then: The expected outcome - the system's new state after the action
 - And: Additional conditions chained to any of the above

Why Gherkin?

Gherkin scenarios can be directly automated using Cucumber, XCTest (iOS), or Espresso (Android). A story with complete Given/When/Then scenarios is 80% of the way to an automated acceptance test suite. Writing them forces precision in requirements before any code is written.



Given / When / Then - Acceptance Criteria

Story: As a field technician, I want to log hours offline so that I can work in areas with no connectivity.

Scenario 1: Logging while offline

Given the device has no network connectivity and the user is on the Time Entry screen

When the user enters 4.5 hours for Project Alpha and taps Save

Then the entry is saved to local storage, and the entry appears in the list with a 'Pending Sync' label

Scenario 2: Sync on reconnect

Given the device has 3 entries with 'Pending Sync' status

When the device regains network connectivity

Then the app automatically syncs all pending entries and successfully synced entries update their status to 'Submitted'



Common User Story Anti-Patterns

Anti-Pattern	Example	Problem	Fix
Technical task masquerading as a story	"As a developer, I want to refactor the network layer so that it uses protocols."	No user value. The 'actor' is the developer, not a system user.	Developers are not story actors. Track technical work as tasks or enablers with clear rationale tied to user value.
Missing business value	"As a manager, I want to see a dashboard."	No 'so that' clause. Why do they want a dashboard? What decision does it enable?	"...so that I can identify which employees have not submitted timesheets before the Friday deadline."
Feature-as-story (too large)	"As a user, I want to manage my account."	Unestimable. Includes update email, change password, update profile photo, deactivate account, set notification preferences - each a separate story.	Split into individual capabilities, each independently valuable.
Prescribes implementation	"As a user, I want a modal dialog that appears when I tap the + button."	Specifies the solution, not the need. Design is not the story author's domain.	"As a user, I want to add a new time entry from the timesheet screen." (Let designers and engineers decide the interaction pattern.)



User Story Writing Workshop

Skill Check ✓

- Rewrite the following Business Requirements as User Stories, complete with acceptance criteria:
- Feature request: 'The app should let managers see who hasn't submitted their timesheet yet.'
- Feature request: 'We need the app to handle when field technicians work in areas with no cell service.'
- Feature request: 'Employees should be able to correct a timesheet after submission if they made a mistake.'



Scrum Ceremonies - Purpose and Anti-Patterns

Ceremony	Time-box	Purpose	Most Common Anti-Pattern
Sprint Planning	4 hrs (2-wk sprint)	Sprint Goal + Sprint Backlog	Planning without a Sprint Goal - just a list of tasks
Daily Scrum	15 min	Inspect progress toward Sprint Goal	Status report to manager instead of peer sync
Sprint Review	2 hrs (2-wk sprint)	Demo Done work to stakeholders, adapt backlog	Showing not-Done work. No stakeholders present.
Sprint Retrospective	1.5 hrs	Improve process - specific actions with owners	12 items with no owners and no due dates



Five Events, One Continuous Loop



Timeboxes (for a 1-month Sprint)

Sprint Planning

8 hrs max

Daily Scrum

15 min - always

Sprint Review

4 hrs max

Retrospective

3 hrs max

Shorter Sprints scale these down proportionally. The timebox is a ceiling, not a target.



Sprint Planning

Sprint Planning answers three questions

WHY is this Sprint valuable?

The PO proposes; the team crafts the Sprint Goal together

WHAT can be done this Sprint?

Developers pull items from the top of the backlog - they choose the amount

HOW will the work get done?

The team plans its own approach. No one plans it for them



The Daily Scrum

The Daily Scrum is NOT a status meeting

15 minutes, daily, for the Developers

Purpose: inspect progress toward the Sprint Goal and adapt the plan

No one reports to anyone - the team coordinates with itself

SM and PO attendance: optional

The 3-question script is a guideline, not a rule - many teams walk the board instead

Each team member answers three questions:

WHAT have you done since yesterday?

WHAT are you planning for today?

WHAT roadblocks are you currently facing?



Sprint Review



Sprint Review

Inspects the **PRODUCT**

- Working Increment shown to real stakeholders
- Feedback flows into the Product Backlog
- Not a polished demo - an honest inspection
- Replaces the “big reveal” at the end of a Waterfall project

Compare: in Waterfall, stakeholders often see the product for the first time at UAT - when change is most expensive.



Sprint Retrospective



Sprint Retrospective

Inspects the **PROCESS**

- How did we work together this Sprint?
- What will we improve - concretely - next Sprint?
- The engine of continuous improvement
- Most skipped when pressure is high - exactly when it's most needed

A Retro that produces the same action items every Sprint isn't a Retro. It's a ritual.



Effort Estimation and Planning Poker

Scrum Does Not Require Story Points

The Scrum Guide
mentions story points...

0 times

Scrum requires that the team can size work well enough to make a credible Sprint forecast. How is entirely up to the team.

When estimation helps

- ✓ Forces conversation that surfaces hidden complexity
- ✓ Builds shared understanding before work starts
- ✓ Supports credible forecasting for stakeholders

When estimation hurts

- ✗ Debating points consumes more time than the work
- ✗ Estimates get treated as commitments - then weaponized
- ✗ Precision theater: false confidence in unknowable numbers



Common Estimation Techniques

Story Points

Relative sizing (Fibonacci). Measures complexity, not hours.

Best for: Teams wanting effort decoupled from time

Planning Poker

Simultaneous reveal prevents anchoring. Discussion on splits.

Best for: Surfacing disagreement that hides complexity

T-Shirt Sizing

S / M / L / XL. Fast, low-argument, roughly right.

Best for: Early grooming and roadmap-level planning

Right-Sizing

Split everything until it's "a day or two." Count items.

Best for: Mature teams who want radical simplicity

Raw Time

Hours or days. Familiar and simple.

Best for: Teams new to Scrum or non-software contexts

Throughput

Forecast from historical items-per-Sprint data.

Best for: Stable teams with consistent item sizes



Story Points - What They Mean in Practice

Points	Meaning	Mobile Example
1	Trivial. Completely understood. No unknowns.	Change an error message. Fix a typo. Update a button color.
2	Small, well-understood. One component.	Add a loading spinner to an existing screen. Show one additional field.
3	Moderate. 2–3 components. Straightforward tests.	New form with validation and API call. Filter option on existing list.
5	Significant. Multiple components. Integration testing needed.	New screen from one API endpoint - loading, error, empty, populated states.
8	Complex. Cross-cutting. Significant unknowns.	Push notification handling with deep link. Offline-capable screen.
13	Very complex. Architecture implications. Multiple unknowns.	Offline sync with conflict resolution. Biometric auth on existing flow.
21	Too large. Must be decomposed before committing.	Any 21-point story should be split. If it won't split, spike first.



Planning Poker - Mechanics and Value

- PO reads the story. Team asks clarifying questions. Questions stop when no new information is coming.
- Each team member privately selects their estimate card.
- All cards revealed **SIMULTANEOUSLY** - this prevents anchoring (the first number heard biases everyone else).
- If estimates are identical or within one Fibonacci value: adopt. Brief discussion.
- If estimates diverge significantly: HIGH estimator explains first, then LOW. Discussion follows.
- Revote after discussion if needed. Typically 1–2 rounds to consensus.
- The most valuable moment: when a 3 and a 13 appear on the table. Don't rush to consensus - the divergence is the information.



7 Estimation Anchors - Questions to Ask Before Voting

Question	Direction if YES
Have we built something similar before?	YES → lower estimate. NO → higher estimate or spike.
Does this require a spike first?	YES → do not estimate this story yet. Spike it first.
Does this touch iOS AND Android?	YES → multiply, don't just add. Platform parity is real cost.
Does this touch multiple services or teams?	YES → add coordination overhead - meetings, waiting, alignment.
Are there security or compliance requirements?	YES → add significant weight. Security review is never free.
Are there performance requirements needing profiling?	YES → add time for measurement and tuning cycles.
Are there accessibility requirements?	YES → add implementation and VoiceOver/TalkBack test time.



Story Point Estimation - Four Stories

Story 1

Add a note field to an existing time entry form.

Story 2

Daily summary push notification for managers.

Story 3

GPS geofencing to auto-detect shift start/end.

Story 4

Configurable overtime threshold per contract type.



Plan your Sprint from this Backlog

Skill Check ✓

#	User Story	Points	Priority	In Sprint 1?
1	As a user, I can log in with SSO credentials	?	Critical	Y / N
2	As a user, I can view my account dashboard	?	High	Y / N
3	As a user, I can update my profile information	?	High	Y / N
4	As an admin, I can assign user roles	?	Medium	Y / N
5	As a user, I can reset my password via email	?	High	Y / N
6	As a user, I can view my transaction history	?	Medium	Y / N
7	As a user, I receive email notifications on order status	?	Medium	Y / N
8	As an admin, I can export user data to CSV	?	Low	Y / N
9	As a user, I can set notification preferences	?	Low	Y / N
10	As a user, I can access help documentation	?	Low	Y / N



Platform Selection Strategy

Native vs. Hybrid vs. Cross-Platform

Approach	Technologies	Pros	Cons	Best For
Native	Swift/Xcode · Kotlin/Android Studio	Max performance · Full API access · Best UX fidelity	Two codebases · Highest cost	Enterprise apps · Complex hardware integration · 5+ year lifespan
Hybrid	Ionic + Capacitor · Cordova	One codebase · Web dev familiarity	Bridge overhead · Limited native APIs · Inconsistent UX	Simple internal tools · Low-interaction apps
Cross-Platform	React Native · Flutter · KMM	Shared logic or UI · Growing ecosystem	Abstraction layers · Platform-specific edge cases	Teams with JS/Dart experience · 3–5 year horizon



Decision Framework

Question	Native Indicator	Cross-Platform Indicator	Hybrid Indicator
What are the performance requirements?	Real-time, animation-heavy, or hardware-intensive	Standard CRUD, moderate interaction	Simple forms, infrequent user interaction
What native APIs are required?	Deep integration: ARKit, HealthKit, CoreBluetooth, Face ID	Standard: push notifications, camera, GPS	Minimal: basic device info, web views
What is the team composition?	Dedicated iOS and Android engineers	Full-stack or JavaScript engineers	Web developers with no mobile experience
What are the UX requirements?	Platform-native conventions mandatory	Good UX acceptable; exact convention match not required	Functional over polished
What is the timeline to market?	Speed is less important than quality and maintainability	Speed matters; code sharing accelerates delivery	Fastest time to market with existing web team
What is the expected app lifespan?	5-10+ years; long-term maintainability critical	3-5 years; framework risk acceptable	1-3 years; experimental or internal tool



Platform Selection

Skill Check ✓

Scenario A

Regional bank. Face ID, Apple Pay, camera deposit, WebSocket balances, 5-year support, 14 native engineers. What do you recommend and why?

Scenario B

Equipment rental company. Internal tool, 200 staff, Samsung tablets only, 3-month timeline, React web dev team. What do you recommend?



SOLID Design Principles

The Root Cause

- All five SOLID principles address the same underlying problem: coupling.
- Tight coupling = changes to one component force changes to another.
- SOLID = loose coupling + high cohesion.



All Five SOLID Principles at a Glance

Principle	Core Idea	Violation Signal	The Fix
S - Single Responsibility	One reason to change	Class has methods serving different 'actors'	Split into focused classes; each owns one concern
O - Open/Closed	Open for extension, closed for modification	Adding a new type requires editing existing code	Define a protocol/interface; add new types by adding new classes
L - Liskov Substitution	Subtypes must be substitutable for their base type	<code>fatalError('not supported')</code> in a protocol conformance	Prefer composition; don't extend what can't be substituted
I - Interface Segregation	No code should depend on methods it doesn't use	Implementing <code>fatalError</code> for unused protocol methods	Split fat protocols into smaller, focused ones
D - Dependency Inversion	Depend on abstractions, not concretions	Class creates its own <code>URLSession/Database</code> in its body	Inject dependencies via initializer; depend on protocols



Single Responsibility Principle

- A class should have only one reason to change
 - i.e. a class should be responsible to one, and only one, actor
- A TimesheetService class may do many things
 - e.g. fetch, create, update, and delete timesheets
 - All those things are the responsibility of one actor
- "If we change how emails are sent, do we also need to modify the TimesheetService"?
 - If the answer is yes, SRP is violated



S - Single Responsibility Violation

```
// VIOLATION: Five distinct reasons to change in one class
class TimesheetManager {
    func fetchTimesheets(employeeId: String) -> [Timesheet] {
        let url = "https://api.internal.com/timesheets/(id)" // hardcoded!
        return parseJSON(URLSession.shared.dataTask(url: url))
    }
    func approveTimesheet(_ t: Timesheet) { ... }

    func exportToCSV(_ timesheets: [Timesheet]) -> String { ... }

    func sendEmail(to: String, body: String) { ... }

    func writeToAuditLog(_ msg: String) { ... }
}
```



S - Single Responsibility Fixed

```
class TimesheetRepository {  
    func fetchTimesheets(employeeId: String) -> [Timesheet] { ... }  
    func updateStatus(_ t: Timesheet, to status: TimesheetStatus) { ... }  
}
```

```
class EmailNotificationService: NotificationService {  
    func sendNotification(to recipient: String, body: String) { ... }  
}
```

```
class AuditLogger {  
    func log(_ message: String) { ... }  
}
```



S - Single Responsibility Fixed

```
class TimesheetApprovalCoordinator {  
    // Depends only on protocols - see DIP  
    init(repository: ..., notificationService: ..., auditLogger: ...) { ... }  
  
    func approveTimesheet(_ t: Timesheet) {  
        repository.updateStatus(t, to: .approved)  
        notificationService.sendNotification(to: t.employee.email, body: ...)  
        auditLogger.log("Approved (t.id)")  
    }  
}
```



O - Open/Closed Principle

- Once a class is tested and deployed, you should make modifications to it by extending its behavior, not modifying its source code.
 - Modifications to existing code risk introducing regression bugs.
- Best mechanism to achieve OCP is abstraction
 - Define an interface (protocol in Swift) describing what some class does
 - Implement variations on the behavior in separate classes
 - Adding new behavior means adding a new implementation, not editing existing ones



O - Open/Closed Violation

```
class PaymentProcessor {  
    func processPayment(amount: Double, type: String) {  
        if type == "credit_card" {  
            // credit card logic  
        } else if type == "bank_transfer" {  
            // bank transfer logic  
        } else if type == "paypal" {  
            // PayPal logic <-- added this month  
        }  
        // Next month: else if type == "crypto" { ... }  
    }  
}
```



O - Open/Closed Fixed

```
protocol PaymentProvider {  
    func processPayment(amount: Double) throws  
}  
  
class CreditCardPaymentProvider: PaymentProvider {  
    func processPayment(amount: Double) throws { /* credit card logic */ }  
}  
  
class BankTransferPaymentProvider: PaymentProvider {  
    func processPayment(amount: Double) throws { /* bank transfer logic */ }  
}  
  
class CryptoPaymentProvider: PaymentProvider { // New type: new class only  
    func processPayment(amount: Double) throws { /* crypto logic */ }  
}
```



O - Open/Closed Fixed

```
protocol PaymentProvider {  
    func processPayment(amount: Double) throws  
}  
  
class PaymentProcessor {  
    private let provider: PaymentProvider  
  
    init(provider: PaymentProvider) { self.provider = provider }  
  
    func processPayment(amount: Double) throws {  
        try provider.processPayment(amount: amount)  
    }  
}
```



L - Liskov Substitution Principle

- Objects of a subtype must be substitutable for objects of their parent type without altering the correctness of the program
- e.g. if you have a method that accepts a parameter of type Bird, you should be able to pass it an Ostrich (which extends Bird)
 - The method should still work correctly even though an Ostrich cannot fly
- This is the most subtle of the five SOLID principles
- Often appears in mobile codebases as a protocol (interface) method that throws a `fatalError()` in some conforming types
- Runtime type-checking is often a sign of a violation



L - Liskov Substitution Violation

```
class Rectangle {  
    var width: Double  
    var height: Double  
    init(width: Double, height: Double) { self.width = width; self.height = height; }  
    func area() -> Double { width * height }  
}  
  
class Square: Rectangle {  
    override var width: Double {  
        didSet { height = width }  
    }  
}  
  
func testArea(_ shape: Rectangle) {  
    shape.height = 4; shape.width = 5;  
    assert(shape.area() == 20)           // FAILS for Square: area() returns 25
```



L - Liskov Substitution Fixed

```
protocol Shape {  
    func area() -> Double  
}
```

```
protocol Resizable {  
    var width: Double { get set }  
    var height: Double { get set }  
}
```



L - Liskov Substitution Fixed

```
struct Rectangle: Shape, Resizable {  
    var width: Double  
    var height: Double  
    func area() -> Double { width * height }  
}
```

```
struct Square: Shape {  
    var side: Double  
    func area() -> Double { side * side }  
}
```



I - Interface Segregation Principle

- No code should be forced to depend upon (implement) methods it does not use
- Prefer many small, specific interfaces over one large general one
 - Fat interfaces force implementing classes to provide no-op implementations of methods they do not need
 - A class that claims it can do something but throws an error when you try to do that thing violates the contract



I - Interface Segregation Violation

```
protocol DataSource {  
    func fetchAll() -> [Record]  
    func fetchById(_ id: UUID) -> Record?  
    func insert(_ record: Record)  
    func update(_ record: Record)  
    func delete(_ id: UUID)  
    func fetchPage(offset: Int, limit: Int) -> [Record]  
    func count() -> Int  
}
```



I - Interface Segregation Violation

```
class ReadOnlyRemoteDataSource: DataSource {  
    func fetchAll() -> [Record] { /* network call */ return [] }  
    func fetchById(_ id: UUID) -> Record { /* network call */ return nil }  
    func insert(_ record: Record) { fatalError("Read-only!") } // violation  
    func update(_ record: Record) { fatalError("Read-only!") } // violation  
    func delete(_ id: UUID) { fatalError("Read-only!") } // violation  
    func fetchPage(offset: Int, limit: Int) -> [Record] { return [] }  
    func count() -> Int { return 0 }  
}
```



I - Interface Segregation Fixed

```
protocol Readable {  
    func fetchAll() -> [Record]  
    func fetchById(_ id: UUID) -> Record?  
}  
  
protocol Writable {  
    func insert(_ record: Record)  
    func update(_ record: Record)  
    func delete(_ id: UUID)  
}  
  
protocol Pageable {  
    func fetchPage(offset: Int, limit: Int) -> [Record]  
    func count() -> Int  
}
```



I - Interface Segregation Fixed

```
class ReadOnlyRemoteDataSource: Readable, Pageable {  
    func fetchAll() -> [Record] { /* network call */ return [] }  
    func fetchById(_ id: UUID) -> Record { /* network call */ return nil }  
    func fetchPage(offset: Int, limit: Int) -> [Record] { return [] }  
    func count() -> Int { return 0 }  
}  
  
class LocalCacheDataSource: Readable, Writeable {  
    func fetchAll() -> [Record] { return [] }  
    func fetchById(_ id: UUID) -> Record { return nil }  
    func insert(_ record: Record) { . . . }  
    func update(_ record: Record) { . . . }  
    func delete(_ id: UUID) { . . . }  
}
```



D - Dependency Inversion Principle

- High level modules should not depend on low level modules
 - Both should depend upon abstractions
- Abstractions should not depend upon details (implementations)
 - Details should depend upon abstractions
- This is the foundation of testability
 - When a class directly instantiates its dependencies, they cannot be replaced in tests
 - This makes it impossible to create true unit tests
- When a class depends upon an abstraction instead, the implementation can be passed in from outside the class
 - The application can inject a real implementation at run time
 - Unit tests can inject a mock implementation for testing purposes



D - Dependency Inversion Violation

```
class LoginViewModel {  
    private let networkService = NetworkService()           // concrete, hardcoded  
    private let analyticsService = FirebaseAnalytics()       // concrete, hardcoded  
  
    func login(email: String, password: String) async {  
        let result = await networkService.post("/auth/login",  
            body: ["email": email, "password": password])  
        analyticsService.track(event: "login_attempted")  
        . . .  
    }  
}
```



D - Dependency Inversion Fixed

```
protocol NetworkServiceProtocol {  
    func post(_path: String, body: [String: Any]) async -> Result<Data, Error>  
}  
  
protocol AnalyticsServiceProtocol {  
    func track(event: String)  
}
```



D - Dependency Inversion Fixed

```
class loginViewModel {  
    private let networkService: NetworkServiceProtocol  
    private let analyticsService: AnalyticsServiceProtocol  
  
    init(networkService: NetworkServiceProtocol, analyticsService: AnalyticsServiceProtocol) {  
        self.networkService = networkService  
        self.analyticsService = analyticsService  
    }  
  
    func login(email: String, password: String) async {  
        . . .  
    }  
}
```



D - Dependency Inversion in Tests

```
class MockNetworkService: NetworkServiceProtocol {  
    var responseToReturn: Result<Data, Error> = .success(Data())  
    func post(_path: String, body: [String: Any]) async -> Result<Data, Error> {  
        return responseToReturn  
    }  
}
```

```
let viewModel = LoginViewModel(  
    networkService: MockNetworkService(),  
    analyticsService: MockAnalyticsService()  
)
```



SOLID Principles Applied

- They are not independent
 - In well-designed code, they reinforce each other
- A codebase that honors all five principles can be changed fearlessly
 - You can add features, modify a service, or swap a dependency without touching unrelated code
 - You do not need to keep a spreadsheet list of "things that might break"



SOLID Principles Audit

Skill Check ✓

The Code

TimesheetManager class in the GitHub repository.

Five Questions

Q1: All violations + harm. Q2: Why is fetchTimesheets untestable? Q3: Slack migration - how many files change? Q4: Design a refactored version. Q5: Write one unit test.



Clean Code and Technical Debt

Meaningful Names - The Rules

Category	Rule	Bad	Good
Variables	Reveal intent - what does this represent?	let d = 86400	let secondsInDay = 86400
Booleans	Read as a true/false assertion	let flag = true	let isAuthenticated = true
Functions	Verb-noun pair - describe what it does	func process()	func fetchApprovedTimesheets()
Magic Numbers	Named constants, always	if attempts > 3 {	if attempts > maxLoginAttempts {
Parameters	Same rules as variables - no single letters	func f(a: Int, b: String)	func calc(hours: Int, name: String)



Functions

- Robert Martin's rules for functions:
- Functions should be small (5-15 lines)
 - If you must scroll, it is too long
- Functions should do one thing
 - If you can extract a meaningful sub function from it, it was doing more than one thing
- One level of abstraction per function
 - Don't mix high-level intent with low-level implementation in the same function
- Functions should have no side-effects
 - A function named `getUser` should not also log to a file or increment a counter
- Prefer fewer parameters. Zero is ideal. One is good. Two is acceptable.
 - Three or more requires strong justification
 - More than three is almost always wrong. Group the values into an object.



Function Violations

// VIOLATIONS: too many parameters, vague name, multiple side-effects

```
func manage(id: String, mode: Int, flag: Bool, count: Int, data: [String: Any]) -> Bool {  
    if mode == 1 {  
        log("Entering manage mode 1")  
        // . . . 40 lines of code  
        sendEmail(to: data["email"] as! String, body: "Done")  
        return true  
    }  
    . . .  
}
```



Function Fixed

// CORRECTED: small, focused, descriptive functions with clear names

```
func approveTimesheet(_ timesheet: Timesheet) {  
    repository.updateStatus(timesheet, to: .approved)  
    notificationService.sendApprovalNotification(for: timesheet)  
}  
  
func sendApprovalNotification(for timesheet: Timesheet) {  
    let message = buildApprovalMessage(for: timesheet)  
    emailService.send(to: timesheet.employee.email, body: message)  
}
```



Comments

- The goal is not to avoid all comments - only those that exist because the code is unclear

Comment Type	Rule	Example
Legal comments	Acceptable. Required for copyright, license, authorship.	// Copyright (c) 2026 PNC Bank
Intent comments (why)	Acceptable. Explains why a decision was made, not what the code does.	// We use 45s timeout here because the Apigee P95 latency under load is 38s (see MOB-412)
Warning comments	Acceptable. Warns future engineers of consequences.	// WARNING: Do not call this method on the main thread - it blocks on network I/O
TODO comments	Use sparingly. Track TODOs in the backlog, not the code.	// TODO: Remove this workaround once iOS 16 support is dropped (MOB-892)
Explanatory comments (what)	Avoid. Rewrite the code instead.	// Increment counter by 1 → <code>i += 1</code> (the code says this)
Noise comments	Never. They add noise and go stale.	// Default constructor → constructor with no parameters
Commented-out code	Delete it. Version control is your history.	// <code>let oldService = LegacyTimesheetService()</code>



Error Handling

- Professional code makes error handling explicit
 - Unhandled errors are production incidents waiting to be discovered
- Never swallow exceptions.
 - An empty catch block is a lie - it tells the runtime (and future readers) that nothing bad happened, when in fact something bad happened and you ignored it.
- Provide context in error messages.
 - 'Error' is not a useful error message.
 - 'Failed to fetch timesheets for employee E12345: connection timed out after 30s' is.
- Distinguish between recoverable and unrecoverable errors.
 - A network timeout is recoverable (retry).
 - A corrupted data store is not (show error, log, alert on-call).
- Error handling should not obscure the happy path.
 - If your function is 60% error handling code, restructure it.



Error Handling Violation

// VIOLATION: swallowing an error silently

```
func fetchTimesheets() -> [Timesheet] {  
    do {  
        return try repository.fetchAll()  
    } catch {  
        return []           // the caller has no idea something went wrong  
    }  
}
```



Error Handling Fixed

// CORRECTED: surface the error explicitly

```
func fetchTimesheets() -> Result<[Timesheet], TimesheetError> {  
    do {  
        let timesheets = try repository.fetchAll()  
        return .success(timesheets)  
    } catch let error as RepositoryError {  
        logger.error("Failed to fetch timesheets: \(error.localizedDescription)")  
        return .failure(.repositoryUnavailable(underlying: error))  
    } catch {  
        return .failure(.unknown(underlying: error))  
    }  
}
```



Code Formatting and Consistency

- Code formatting is a team concern, not an individual preference
 - A codebase that uses different formatting conventions in different files - because each engineer has their own preferences - imposes cognitive overhead on every reader and produces noisy diffs that obscure real changes
- The solution is to automate formatting enforcement:
 - iOS: SwiftLint for linting (error and warning rules); swift-format for automated formatting
 - Android: ktlint for Kotlin linting and formatting; Android Studio's built-in formatter
- Run formatters and linters in the CI pipeline - a build that fails lint is not mergeable
- Never debate formatting in code review
 - If the linter accepts it, it's acceptable. If it doesn't, fix it.



Technical Debt - Fowler's Quadrant

- Like financial debt, technical debt is a useful tool when used deliberately and paid back promptly
 - It is also a dangerous liability when it accumulates uncontrolled

	Prudent	Reckless
Deliberate	"We'll ship now and refactor post-launch" - acceptable if tracked and paid	"No time for design" - shortcuts with no plan to pay back
Inadvertent	"Now we see how we should have done it" - normal learning; address promptly	"What is layered architecture?" - absent knowledge that should have been present



Common Sources of Technical Debt in Mobile Engineering

Source	Description	Compounding Effect
Missing tests	Features shipped without unit tests. Refactoring the code later is risky because there is no safety net.	Each refactor without tests risks breaking existing behavior silently. Over time, engineers stop refactoring out of fear. Code solidifies in bad states.
Copy-paste programming	Logic duplicated because extraction was considered too slow. The same bug now lives in ten places.	Every bug fix must be applied to every copy. Every feature change must be propagated. The team loses track of which copy is authoritative.
God classes	A single class (often named Manager, Handler, or Controller) that knows everything and does everything.	Every feature request touches the god class. Merge conflicts on this file are constant. Testing is nearly impossible because the class has hundreds of dependencies.
Outdated dependencies	Third-party libraries or platform SDKs that have not been updated. Security vulnerabilities accumulate; API deprecations accumulate.	When a critical security fix requires a major version upgrade, the accumulated API changes make the upgrade a weeks-long project instead of a day's work.
No API versioning strategy	Mobile apps talk directly to v1 of an API with no migration plan. When the backend team changes the API, every app in the field breaks.	Each backend change requires a coordinated forced-update campaign to ensure users are on the new app version - which many users resist.
Inadequate error handling	Errors swallowed silently or handled with generic fallbacks. Production issues are invisible until users complain.	Error rates climb without triggering alerts. The team learns about bugs from support tickets, not monitoring dashboards. Root cause analysis becomes guesswork.



Measuring Technical Debt

- You cannot manage what you cannot measure.
- Code complexity metrics: Cyclomatic complexity measures the number of independent paths through a function. A function with cyclomatic complexity above 10 is difficult to test and understand. Static analysis tools (SonarQube, SwiftLint rules) can report this automatically.
- Test coverage: The percentage of code lines executed by at least one test. Coverage below 60% on a codebase in active development is a red flag. Coverage above 80% on the core business logic layer is a reasonable target.
- Code duplication percentage: Tools like PMD CPD (Copy-Paste Detector) identify duplicated code blocks. High duplication percentages indicate violation of the DRY (Don't Repeat Yourself) principle.
- Dependency staleness: The average age of third-party dependencies relative to their current release. Anything more than two major versions behind warrants investigation.



Managing and Paying Down Technical Debt

Strategy	Description	When to Apply
Boy Scout Rule	Every time you touch a file, leave it slightly cleaner than you found it. Rename an unclear variable. Extract a long method. Add a missing test for a function you read.	Every pull request, every day. Not a separate activity - a continuous habit.
Debt backlog	Maintain a section of the product backlog dedicated to debt reduction work. Each item is described, estimated, and prioritized alongside features. It is visible to the Product Owner.	Whenever a debt item is significant enough to warrant dedicated sprint capacity.
Dedicated refactoring sprints	Occasionally run a sprint (or part of a sprint) focused entirely on technical debt reduction. No new features. Only cleanup, testing, and architecture improvements.	When debt has accumulated to the point of materially slowing feature delivery - typically quarterly or semi-annually.
Incremental strangler pattern	For large rewrites, incrementally replace legacy code by routing traffic through a new implementation while the old one is still running. New and old coexist until the migration is complete.	When an entire component must be replaced but cannot be taken offline for a full rewrite.
Hard DoD requirements	Enforce code coverage thresholds, linting rules, and complexity limits in the CI pipeline. Builds that violate these requirements fail. Debt cannot be merged.	Continuously, from the start of a project. Retrofitting these requirements on an existing codebase is painful - preventing debt is easier than curing it.



Knowledge Check

Knowledge Check · Question 1

Describe the difference between functional and non-functional requirements. Give two examples of each for a mobile timesheet application.

Knowledge Check · Question 2

A product manager says: 'We are an Agile team, so we don't need design documentation.' How do you respond with a precise, evidence-based correction?

Knowledge Check · Question 3

Explain the Dependency Inversion Principle. Why does it directly enable unit testing, and what specific technique implements it?

Knowledge Check · Question 4

A team's Sprint Review reveals 4 of 7 committed stories are not Done. The Scrum Master says: 'We'll carry them over and count them in next Sprint's velocity.' What is wrong with this approach?

Knowledge Check · Question 5

What is the Test Pyramid? Why does an 'inverted pyramid' (many UI tests, few unit tests) indicate a maintenance problem?